

Stabilizing discrete GPU Unified Memory Performance Under Oversubscription.

Senad-Leandro Lemes Galera
Technical University Munich
Munich, Germany
senad.lemes@tum.de

ABSTRACT

Unified Memory (UM) simplifies GPU programming by enabling transparent memory oversubscription. However, when the working set exceeds available VRAM, on-demand paging can cause severe performance instability. Developers must then choose between manual chunking with pinned memory, mapped (zero-copy) host memory, or UM. UM handles the memory management under the hood. Naively, UM implements an on-demand tree-based neighborhood (TBN) prefetcher. Eviction is implemented using an age-based LRU policy. Unfortunately, this unoptimized UM leads to a performance collapse beyond device capacity. Under irregular and random access patterns, collapse is caused by a combination of (i) on-demand paging (ii) frequent evictions, which together amplify migration thrashing. Across sequential, ranged, and random analytical workloads whose working sets exceed VRAM, naive UM incurs up to $74\times$ slowdown. Our results show that aggressive prefetching can match pinned memory performance for memory-resident workloads. Furthermore, for larger workloads, leveraging zero-copy alongside internal UM optimizations provides superior performance compared to traditional memory mapping. UM allows us to integrate both approaches, delivering competitive performance without sacrificing the simplicity of its abstraction layer. We propose a hybrid memory management model combining aggressive prefetching with zero-copying data when exceeding device memory. Across UVM-benchmarks our approach showed a geo-mean improvement of $10.03\times$ in comparison to the TBN-prefetching scheme. When oversubscribing, our provided migration scheme outperformed mapped memory by up to $10\times$ while remaining partially lower-bounded by mapped memory performance.

PVLDB Reference Format:

Senad-Leandro Lemes Galera. Stabilizing discrete GPU Unified Memory Performance Under Oversubscription.. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Despite the emergence of architectures with physically unified memory, discrete GPUs with separate CPU and GPU memory remain one of the dominant platforms in modern datacenters [5, 7, 19, 44]. For many data-intensive workloads, this separation becomes critical once the working set exceeds available GPU memory (VRAM). In such cases, performance depends on how efficiently data can be accessed or moved between host memory and GPU memory.

To reduce the complexity of manually managing data movement between CPU and GPU memory, Unified Memory (UM), implemented by NVIDIA as Unified Virtual Memory (UVM) [1], was introduced. It provides a unified virtual address space across CPU and GPU memory. This abstraction promises independence from explicit data movement and performance stability across workload sizes and types. This removes the burden of reasoning about data placement from the programmer. In addition, it enables applications to oversubscribe VRAM.

However, the currently implemented data movement strategy can lead to significant performance drops when oversubscribing VRAM [3, 21, 26, 46]. The underlying mechanism relies on an on-demand Tree-Based-Neighborhood (TBN) [13] prefetcher, that speculatively prefetches spatially adjacent pages of up to 2 MiB chunks. Eviction is implemented using an age-based LRU mechanism, which evicts up to 2MiB chunks. For complex workloads whose working sets exceed VRAM capacity, on-demand prefetching in chunks of at most 2 MiB, combined with equally small-sized eviction, is often insufficient to hide the high latency of GPU page faults [45]. Because of this performance variability, UVM has seen limited adoption in performance-critical settings [39].

Developers often fall back on alternative memory management strategies due to unpredictable performance. Manual chunking combined with pinned memory can provide high performance, but it introduces additional complexity for the programmer and lacks generalizability across programs. Another option is zero-copy, which maps CPU memory into the GPU address space. This stabilizes the performance, but it is now interconnect-bound (e.g., PCIe or NVLink), which leads to performance drop by multiple orders of magnitude when our workload fits in VRAM.

To address this performance variability, UVM exposes control mechanisms that allow programmers to influence memory behavior explicitly. In particular, it provides asynchronous prefetching and memory advice, the latter allowing programmers to express hints about placement decisions and anticipated access patterns. Using these mechanisms effectively without sacrificing the simplicity of the abstraction remains an open challenge [2, 24]. To address this performance variability, UVM exposes control mechanisms

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

that allow programmers to influence memory behavior explicitly. In particular, it provides asynchronous prefetching and memory advice, the latter allowing programmers to express hints about placement decisions and anticipated access patterns. Using these mechanisms effectively without sacrificing the simplicity of the abstraction remains an open challenge [2, 24].

The prefetching mechanism enables proactive migration of data from host memory into GPU HBM, while memory advice can enable zero-copy access to host-resident pages. We therefore view UVM as a hybrid memory model that combines two complementary modes of use: migration into device memory and mapped access from host memory. Exploiting UVM in this manner is unconventional, because these two modes are typically treated as separate design strategies. Combining them effectively requires explicit decisions about which data should be migrated into HBM and which data should remain host-resident and be accessed through mappings.

Our main contribution is a memory management strategy, which is generalizable across a variety of workloads and reduces the number of page faults through a combination of aggressive prefetching and zero-copy access. We stabilize performance even under complex workloads exceeding VRAM size, while preserving the simplicity of UVM through a small set of rules and maintaining its core goal of abstracting away manual data movement.

Our paper makes the following contributions:

- (1) **A systematic empirical characterization** of UM behavior under oversubscription across diverse workload classes, identifying the specific conditions that trigger performance collapse.
- (2) **An optimized migration strategy** demonstrating that combining large-chunk, usage-aware prefetching with zero-copy memory advice effectively eliminates on-demand page thrashing and stabilizes execution time.
- (3) **Performance validation** showing that this optimized UM approach outperforms mapped memory by up to 10× under oversubscription, and achieves performance parity with manual pinned-memory management when datasets fit within VRAM.
- (4) **A principled, reproducible programming framework** that enables predictable GPU memory management across varying workload sizes without requiring modifications to existing hardware or runtime environments.

2 BACKGROUND

In this section, we first present the three different memory models. In the end, we look at related work.

2.1 Unified Virtual Memory (UVM)

UVM was introduced by NVIDIA in 2013 with CUDA 6.0. UVM offers a virtual address space between the GPU and CPU. It allows oversubscribing VRAM. The underlying data movement is handled automatically, reducing the complexity from a programmer’s perspective. However, this automatic page fault handling introduce significant challenges that can lead to severe performance drops. To address this, while UVM handles data migration automatically by default, CUDA also exposes two mechanisms that allow programmers to guide placement and movement decisions:

cudaMemPrefetchAsync and *cudaMemAdvise*. We can provide these advises to give the compiler some hints about where data is placed and how we will access it, which is analogous to the Linux *madvise()* syscall, where users hint at expected access patterns without altering program correctness. There are three possible advises:

- (1) {Unset|Set}ReadMostly
- (2) {Unset|Set}PreferredLocation
- (3) {Unset|Set}AccessedBy

According to the CUDA documentation [34].

(1) creates a read-only copy of the pages in the target’s processor memory. If we combine it with prefetching, a read-only copy is also created on the destination memory.

(2) guides the migration policy. We will migrate data, if it is not in it’s preferred location or we cannot establish a direct mapping.

(3) this advice causes the data to always be mapped into the target page table as long as a mapping is permitted. If we migrate the data, e.g., via a prefetch, the page table is updated accordingly.

CUDA’s automatic page fault handling combines two mechanisms: on-demand fault resolution, and TBN-prefetching.

On-demand page fault handling NVIDIA’s UVM implements on-demand page fault handling. This means that page faults are handled eagerly, the system only resolves them when they occur.

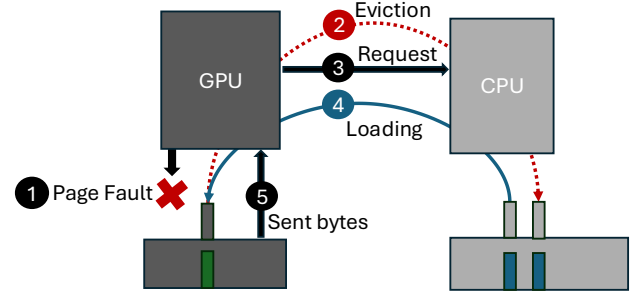


Figure 1: How does the on-demand page faulting works when our VRAM is oversubscribed.

In Figure 1, we can see the process of page faulting when we oversubscribe our VRAM. **1** A page fault occurs in our system, so we do not have the needed page in our VRAM. **2** We need to evict a page from our VRAM. We do that by first unmapping the page from VRAM, then sending the page over the interconnect to our CPU and mapping the page back into the CPU RAM. **3** After correctly evicting the page, we are now able to resolve the page fault. We send a page request over our interconnect to the CPU. **4** To resolve the page request, we need to load the page from the CPU. First, we unmap the page from the CPU. Second, we sent the page over the interconnect to the GPU. Last, we map the page into GPU memory. **5** After the page is resident in the GPU memory, we can now send the requested data to the Streaming Multiprocessor (SM).

When the VRAM is not oversubscribed we can just remove **2** in Figure 1. We only need to request the page, then load it to the GPU memory and in the end sending the bytes to our SM.

TBN-prefetching To make the on-demand page handling more effective, CUDA implements its own prefetching mechanism. It uses a tree-based neighborhood (TBN) prefetching. The following explanation and Figure 2 are based on prior work from Ganguly et al. [13].

The mechanism can be understood as a tree-based policy over memory blocks managed by the GPU Memory Management Unit (GMMU). The allocation is partitioned into fixed-size leaf blocks (e.g., 64 KB), which form the leaves of a hierarchical tree. Each internal node stores a to-be-valid size, i.e., the amount of memory within its subtree that is expected to reside on the GPU after migration.

When a page fault occurs in a leaf block, the corresponding leaf is marked as to-be-valid. For example, if a fault occurs within a 64 KB block, the entire 64 KB leaf is counted as to-be-valid. This information is then propagated upward through the tree by updating the accumulated to-be-valid size of all ancestor nodes.

Prefetching is triggered when the to-be-valid size of a node exceeds 50% of that node’s total size. Once this threshold is crossed, the remaining blocks in that subtree are marked for prefetching by recursively traversing the node’s children. In this way, isolated page faults can progressively activate larger prefetch regions.

Figure 2 illustrates this mechanism. The first page fault, labeled (1), occurs in node D1. As a result, the entire 64 KB block represented by D1 is marked as to-be-valid. This update is propagated to its parent, C1. Since the accumulated to-be-valid size in C1 now exceeds 50% of C1’s total size, the sibling block D2 is also marked as to-be-valid. The second page fault, labeled (2), causes the same process to occur in C2, eventually making C2 fully to-be-valid as well. At that point, B1 becomes completely to-be-valid. Once the accumulated to-be-valid size of node A exceeds the 50% threshold, the same rule is applied to the other subtree B2. As a consequence, all blocks under A are marked as to-be-valid, and the entire region represented by A is prefetched.

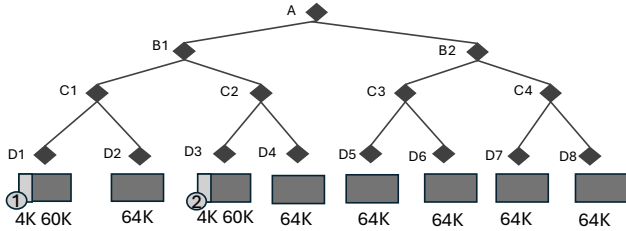


Figure 2: TBN prefetching mechanism. Explanation based on the work by Ganguly et al.[13].

2.2 Pinning Memory

Pinned memory refers to page-locked host memory that cannot be paged out by the operating system and therefore supports efficient direct transfers between CPU memory and GPU memory. Pinned allocations are commonly used as staging buffers for explicit data movement, allowing the runtime to perform high-bandwidth DMA transfers between host DRAM and device memory. Compared to pageable host memory, pinned memory reduces transfer overhead

and enables overlap of communication and computation through asynchronous memcopy operations.

However, data must still be explicitly partitioned and moved by the programmer, which increases implementation complexity and makes performance strongly dependent on how well the transfer schedule matches the workload. This is one disadvantage compared to UVM. Furthermore, unlike UVM, it is not possible to oversubscribe a standard pinned memory allocation.

2.3 Mapping Memory

Mapped memory allows the GPU to directly access host memory by mapping CPU pages into the GPU address space instead of migrating them into device memory, which is why it is often referred to as zero-copy.

In this model, data remains physically resident in system memory, while the GPU issues memory requests over the CPU-GPU interconnect, such as PCIe or NVLink. As a result, zero-copy avoids explicit data transfers and eliminates page migration overhead, such as eviction, which can improve robustness when the working set exceeds available VRAM. Additionally, we completely remove the expensive page eviction.

However, because accesses are served from host memory rather than high-bandwidth device memory, performance is typically limited by interconnect bandwidth and latency, making zero-copy substantially slower than native GPU-memory access for memory-resident workloads.

2.4 Related Work

Prior work has shown that naive use of Unified Memory can lead to significant performance variability due to reactive, page-granular migration and redundant data movement [14, 18, 28, 33]. To mitigate these effects, subsequent efforts adapt either the communication runtime or the application itself. For example, Manian et al. [31] introduce UM-aware MPI designs to reduce unnecessary host staging, while Gera et al. [17] restructure graph traversal algorithms to better align access patterns with UM behavior.

However, these approaches primarily compensate for the reactive nature of Unified Memory rather than systematically exploiting its memory model to control data placement and migration. As a result, performance improvements often require substantial runtime or algorithmic modifications [22, 30, 32, 40] instead of leveraging UM’s abstraction directly.

To address this performance variability, UVM exposes control mechanisms that allow programmers to influence memory behavior explicitly. In particular, it provides asynchronous prefetching and memory advice, the latter allowing programmers to express hints about placement decisions and anticipated access patterns. Using these mechanisms effectively without sacrificing the simplicity of the abstraction remains an open challenge [2, 24]. There is some existing work that exploits UM abstractions. For example, Awan et al. [6] propose a framework that optimizes DNN workloads. However, they do not propose a general framework; instead, they focus on a specific workload.

Allen et al. [4] correctly recognize that on-demand page faulting combined with LRU-based eviction in cases where we oversubscribe can lead to a huge performance drop.

Kim et al. [26] address oversubscription for irregular workloads by identifying inefficiencies in GPU page-fault handling and coordinating execution with page replacement to reduce demand-paging overhead. More recently, Jones et al. [24] characterize Unified Memory access patterns under oversubscription and uses this information to improve placement decisions. Both approaches share our observation that naive on-demand paging is insufficient under memory pressure and provide a schema to conquer this problem.

However, none of these works treat UVM as a hybrid memory model whose migration- and mapping-based modes can be combined within a unified strategy. As a result, there is still no simple and generalizable approach that avoids complex metrics and run-time or algorithmic optimizations, and instead relies on a principled framework to stabilize performance across workloads.

3 PAGE MIGRATION IN A UNIFIED MEMORY ENVIRONMENT

We hypothesize that TBN’s on-demand, small-chunk migration is effective only when the access pattern aligns with its spatial prefetching behavior, and that its limitations become more pronounced once the working set exceeds VRAM capacity. To evaluate this hypothesis, we study three workload classes: sequential, range-based, and random access patterns.

To make these limitations visible, we compare TBN against three alternative strategies that we develop in more detail later: *Large-Chunk Prefetching (LCP)*, which proactively migrates portions as large as possible of the working set, and *memAdvise*, which uses *cudaMemAdvise* to bias UVM toward mapped access behavior, and the combination of both strategies. Their full design is presented in Section 4 and the corresponding discussion below. All experiments use CUDA Unified Memory, and all allocations are created with *cudaMallocManaged*.

3.1 Sequential Access Patterns

For sequential workloads, TBN’s neighborhood-based prefetching should be relatively well matched to the access pattern as long as the working set fits into VRAM. Once the working set exceeds VRAM capacity, however, we expect this advantage to diminish, since additional evictions and on-demand page faults should increasingly dominate performance.

To evaluate this behavior, we construct a sequential microbenchmark and compare TBN against three alternative strategies: zero-copy via our advising strategy, LCP, and LCP combined with zero-copy. We evaluate element sizes ranging from 1 B to 8 B. For the smallest element size, zero-copy performs noticeably worse than the migration-based approaches (Fig. 3(a)), because PCIe transfer overhead is no longer negligible at such fine access granularity. In this case, operating at page granularity is more efficient. This behavior is largely an edge case, however, since most practical workloads use larger element sizes.

The main result is that LCP outperforms all other approaches as long as the working set fits entirely into VRAM (Fig. 3(a)–(d)). In this regime, the overhead of proactive data migration is outweighed by the benefit of serving accesses from HBM. Once the working set exceeds VRAM capacity, however, zero-copy, either alone or combined with LCP, achieves the best performance. This transition

is expected: after oversubscribing VRAM, data migration can no longer be amortized effectively, and the additional need to evict data introduces substantial overhead. Zero-copy avoids both costly page-fault handling and migration overhead and therefore performs best in the oversubscribed regime.

The reason why LCP loses its advantage beyond VRAM capacity can be seen directly in Fig. 5 and Fig. 6. As soon as the VRAM limit is reached, the number of page faults rises sharply and approaches the level observed for TBN. At the same time, the eviction volume increases substantially once the working set exceeds VRAM. In our measurements, LCP evicts up to twice as much data as TBN, indicating considerable unnecessary data movement. This is caused by large prefetches that are not always issued in the optimal order. As a result, once the working set exceeds VRAM, LCP effectively collapses toward TBN because page-fault handling is again delegated to CUDA. This explains why the runtimes of TBN and LCP become similar in the oversubscribed regime.

Overall, the sequential workload confirms that large-chunk prefetching is highly effective when the full working set can reside in VRAM, but that this advantage vanishes once oversubscription forces execution back into a page-fault-driven regime. This shows that prefetching strategy alone is insufficient under oversubscription and that eviction behavior is an equally important bottleneck.

3.2 Range Access Patterns

For range-based workloads, we expect behavior similar to the sequential case, since accesses still exhibit locality within individual ranges. At the same time, because the workload touches only a subset of the data, thus less data has to be read over interconnect, the trade-off between migration and mapped access should become more pronounced. In particular, we expect hybrid strategies to benefit from keeping useful pages in VRAM while tolerating some interconnect-based accesses for the rest.

To evaluate this behavior, we construct a range-based microbenchmark and compare TBN against zero-copy via our advising strategy, LCP, and LCP combined with zero-copy. Figure 4 shows the performance for different range widths. Overall, the results follow the same broad trend as in the sequential case (Fig. 3), but the reduced number of accessed elements changes the relative trade-offs between migration and mapping.

The main finding is that *memAdvise* combined with LCP performs best across much of the evaluated range-based regime. Compared to the sequential workload, this can be explained by the fact that the benchmark touches far fewer elements overall, so accesses served over the interconnect contribute less to total runtime. As a result, the cost of leaving some pages mapped becomes smaller, while retaining useful pages in VRAM through LCP still provides the benefit of HBM access. At the same time, the overhead introduced by memory advice becomes negligible once the working set exceeds VRAM capacity and therefore does not lead to substantial performance degradation in the oversubscribed regime.

When the working set fits entirely into VRAM, however, LCP alone still outperforms both mapping and TBN (Fig. 4). In this regime, migrating the full working set into GPU memory and serving accesses through HBM remains the best strategy. For these in-VRAM cases, the overhead of memory advice is not negligible,

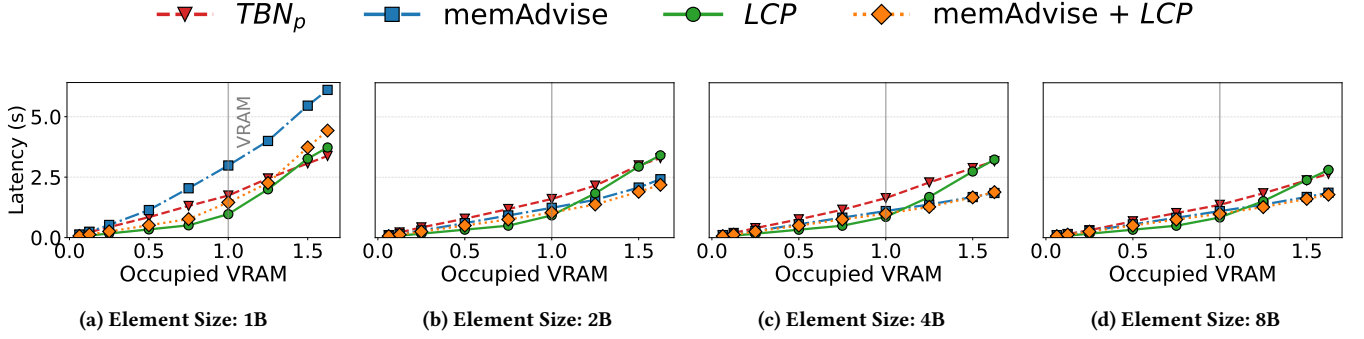


Figure 3: Sequential Access Workload. We touch every element. Subfigures (a)-(d) show increasing element sizes. All plots are scaled to the same y-axis for direct comparison.

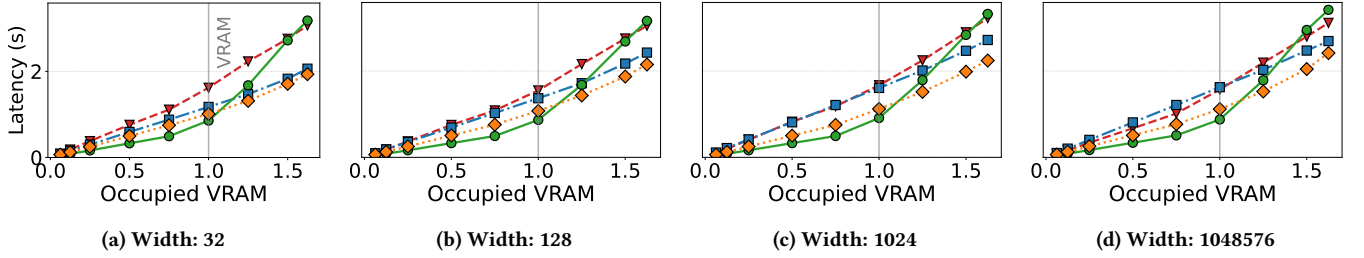


Figure 4: Range Access Workload. Subfigures (a)-(d) show increasing stride widths. All plots are scaled to the same y-axis for direct comparison.

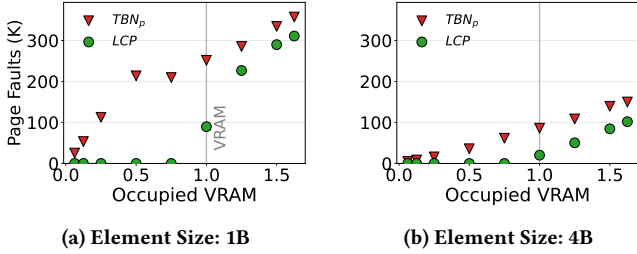


Figure 5: Page Fault Comparison for Sequential Workload.

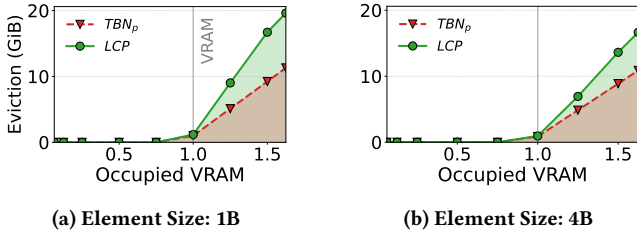


Figure 6: Eviction Size Comparison for Sequential Workload.

so LCP performs better than memAdvise combined with LCP. This overhead is small but measurable, since each memory advice request is implemented through an `ioctl` system call. In our configuration, the advising strategy issues two such calls.

Unlike the 1 B sequential case in Fig. 3(a), the performance drop of zero-copy is not observed here. The reason is that the range-based benchmark operates on 4 B elements, for which PCIe message overhead is no longer a dominant factor. Consequently, mapping remains a comparatively stable strategy across all evaluated range-based workloads.

TBN performs reasonably well for these microbenchmarks, but it is consistently outperformed by at least one alternative strategy, with slowdowns ranging roughly from 10% to 50%. This indicates that although TBN can exploit the remaining locality in range-based accesses, its native prefetching behavior is still not sufficient to achieve the best available performance. Overall, the range-based workload confirms that partial locality alone does not make TBN competitive with more deliberate migration or hybrid placement strategies. For performance-critical applications, relying solely on CUDA’s native TBN prefetcher is therefore insufficient.

3.3 Random Access Patterns

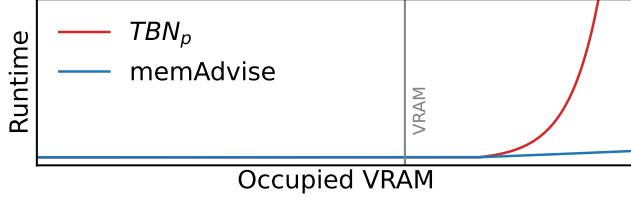


Figure 7: Random Workload. Bitonic Sort Implementation. Abstract View on how the TBN Prefetching approach penalizes runtime when oversubscribing. Lower means better.

3.4 Random Access Patterns

For random access workloads, we expect TBN to perform poorly, since its neighborhood-based prefetching relies on spatial locality that is absent in this regime. Once the working set exceeds VRAM capacity, this mismatch should become even more severe, because prefetched pages are less likely to be reused before eviction and on-demand fault handling should increasingly dominate execution time.

To evaluate this behavior, we use a highly optimized bitonic sort as a bandwidth-intensive workload with structured but irregular memory accesses. Our implementation builds on prior work on CPU vectorization and multi-threaded bitonic sorting [9, 10, 16, 23] as well as GPU-optimized and hybrid designs [20, 38, 43]. The resulting hybrid CPU-GPU implementation sorts 3 billion 32-bit keys in 4.7 seconds, including data transfer, and therefore provides a realistic large-scale stress test for Unified Memory.

The main result is that TBN reaches its limits once the workload exceeds device memory (Fig. 7). In the oversubscribed regime, TBN incurs a slowdown of up to 50× compared to zero-copy. The two main causes of this degradation are the migration of prefetched data that is not reused and the large number of small evictions triggered by oversubscription. Together, these effects produce a migration pattern that is fundamentally mismatched to irregular access behavior.

Figure 8 provides an abstract view of this execution pattern. PCIe bandwidth is utilized primarily during the TBN migration phase and during eviction. The latter is particularly costly, since device-to-host transfers performed for eviction do not contribute useful work and instead stall progress. As a result, TBN fails to utilize the interconnect efficiently and repeatedly interrupts execution with eviction overhead. Zero-copy avoids this behavior: it sustains interconnect-driven execution without repeated migration and eviction phases and therefore performs substantially better in this regime (Fig. 7).

This result stands in clear contrast to the sequential case. TBN performs reasonably well for sequential workloads because its prefetching behavior effectively follows the access stream and migrates data that is likely to be reused soon. It also evicts only as much data as necessary to free space in VRAM, which keeps migration relatively efficient (Fig. 6). For random workloads, however, these locality assumptions break down. Prefetched pages are less likely to be reused before eviction, while irregular accesses trigger

substantially more page faults and increase the cost of on-demand fault handling. Consequently, TBN becomes much less efficient for random workloads than for sequential or range-based access patterns.

Overall, the random workload confirms the central limitation of TBN most clearly: once spatial locality is absent, its on-demand, small-chunk migration strategy becomes a severe bottleneck under oversubscription. In this regime, zero-copy outperforms TBN by orders of magnitude, and combining our advising strategy with LCP can improve performance further. For irregular, oversubscribed workloads, relying on CUDA’s native TBN prefetcher is therefore not a viable performance strategy.

3.5 Limitations of TBN Prefetching

Across sequential, range-based, and random workloads, TBN shows that on-demand paging remains highly sensitive to both access pattern and working-set size. While TBN can perform reasonably well when its neighborhood-based prefetching aligns with the workload, its performance degrades once locality weakens or the working set exceeds VRAM capacity. In particular, Fig. 8 illustrates why TBN fails under oversubscription for workloads with frequent page faults and random access: migration and eviction consume interconnect bandwidth without translating efficiently into useful progress. Taken together, our results reveal three main limitations of the on-demand TBN prefetching scheme:

- (1) Low Throughput (<VRAM)
- (2) On-demand Page Faulting (<VRAM)
- (3) Age-Based small Chunk Eviction (>VRAM)

4 LARGE CHUNK PREFETCHING AND ADVISING STRATEGY

4.1 Large Chunk Prefetching (LCP)

In section 3.5, we named the three big problems with the current UVM implementation. We will use LCP to tackle problem (1), Low Throughput, and (2) On-demand Page Faulting.

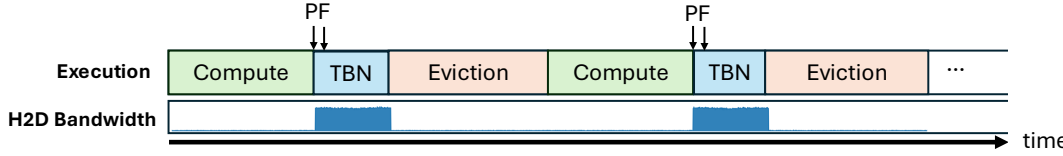
Our prefetching scheme follows the following three rules:

- Prefetch from most to least important data
- Prefetch as much as possible
- Prefetch should never exceed VRAM

Our prefetching scheme leads to three properties, which directly tackle problem (1) and (2). In the end we will shortly show how using LCP can increase performance of an existing application. **High Throughput** Prefetching in hindsight in combination with large chunk prefetching leads to a high throughput. Of course this throughput leads to an start overhead but this overhead is neglectable also seen in Fig. 3 and Fig. 4.

Competitive Performance against Pinned Memory To get competitive performance against pinned memory when being smaller than VRAM we will prefetch the whole dataset to the GPU. This gives us competitive performance against Pinned Memory which is the holy grail when being smaller than VRAM.

No on-demand paging On-demand paging in general is really expensive. Due to the high latency of resolving GPU page faults [45]. Prefetching Large amount of data decreases the amount of page faults drastically (Fig. 5).



There is some existing work that exploits UM abstractions. For example, Awan et al. [6] propose a framework that optimizes DNN workloads. However, they do not propose a general framework; instead, they focus on a specific workload.

Figure 8: Theoretical H2D PCIe Bandwidth of TBN_p Approach. PF = Page Faults. Shows Execution Pattern when Oversubscribing VRAM. TBN_p is not able to utilize full PCIe Bandwidth and also spends unnecessary time evicting pages.

Technique	Workloadsize > VRAM	Workloadsize < VRAM	VRAM Locality	Migration O/head	Low Complexity	Perf. Stability
TBN	+	++	+	+	+++	+
Mapping	++	+	+	+++	+++	++
HELM	++	++	++	++	+	++
Our	+++	+++	++	++	++	+++

Table 1: Qualitative comparison of GPU memory management techniques. Green indicates favorable properties, yellow moderate trade-offs, and red unfavorable characteristics. We focus mainly on compute-heavy workloads.

Evaluation In EMOGI [32], the authors use UM but do not integrate prefetching into their implementation, leaving substantial performance untapped.

To show how our LCP optimization improves the performance we looked at synthesized graphs and real world graphs and how our LCP optimization can improve performance. We also looked at workloads from the SNAP [29] dataset. To get some real world use graphs. We looked at the LiveJournal graph and our optimization improved performance by 5x in comparison to the EMOGI baseline.

Additionally, we used some synthesized graphs to show our performance increase. We were able to increase performance up to 200x of the Emogi [32] Implementation by adding some trivial prefetching.

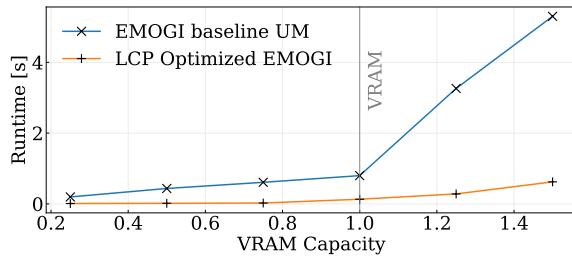


Figure 9: EMOGI graph run with their used UM and our optimized LCP UM. Performance Improvements when < VRAM up to 200x and when > VRAM up to 10x. Run on synthesized graph with edges/node ratio 230.

4.2 Limitations of LCP

Our LCP approach performs well for workloads whose working sets fit within VRAM (Figures 3 and 4). However, once the working set exceeds VRAM capacity, LCP effectively degenerates to the TBN approach. As a result, it again suffers from the same issues:

on-demand page faulting, age-based eviction of small chunks, and low throughput. Prefetching chunks larger than VRAM is ineffective, as it would introduce substantial eviction overhead. Manually prefetching smaller chunks is a possible alternative, but it would significantly increase implementation complexity. Moreover, this strategy would effectively pin memory manually, negating many of the benefits of Unified Memory (UM).

4.3 Advising Strategy

Memory advice can be used to provide the Unified Virtual Memory (UVM) system with hints about data placement and access behavior. In our approach, we use `cudaMemAdviseSetAccessedBy=GPU` and `cudaMemAdviseSetPreferredLocation=CPU` to map CPU-resident memory into the GPU address space, enabling host-memory access without explicit data migration (i.e., zero-copy access).

We use this strategy to address problem (2), on-demand page faulting, and problem (3), age-based eviction of small chunks.

Eliminating on-demand paging. Because each value is accessed directly over the interconnect, no data migration between CPU and GPU is required. Instead, reads and writes occur at fine granularity. This fine-grained access pattern is particularly important for random workloads, where migrating an entire page to access only a small subset of its contents can lead to severe thrashing.

Competitive performance relative to mapped memory. Our advising strategy is conceptually similar to directly mapped host memory. However, our results show that it can outperform the mapped-memory kernel runtime in some cases. More broadly, its performance remains competitive with mapped memory, as both approaches rely on similar underlying mechanisms.

Avoiding eviction. Under zero-copy access, values are fetched directly into the device cache line, eliminating the need for page eviction. This addresses a key limitation of the TBN implementation, where servicing a page fault may trigger additional evictions and, consequently, severe performance degradation.

Evaluation. To evaluate the impact of our advising strategy, we compare against the strategies proposed by Chien et al. [11] and use their implementations as baselines. Figure 10 shows that even small changes in the advising strategy can have a substantial performance impact. For the FDTD3d and BlackScholes benchmarks, our optimized strategy reduces runtime by up to 3×. More generally, the results demonstrate that our approach, when combined with appropriate prefetching, can outperform several alternative advising strategies.

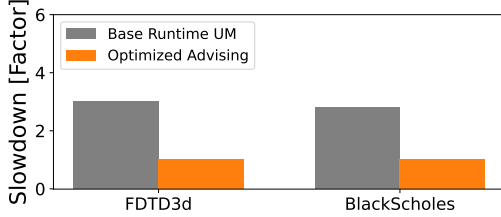


Figure 10: Performance impact of the optimized advising strategy relative to the baseline proposed by Chien et al.

4.4 Limitations of the Advising Strategy

The main limitation of this advising strategy is that its performance degrades when the dataset fits entirely within VRAM. Arithmetic-intensive workloads benefit substantially from placing data in GPU memory and accessing them directly over HBM. Consequently, relying exclusively on this strategy introduces a trade-off between stability and peak performance: while it improves robustness for oversubscribed workloads, it becomes interconnect-bound and therefore underperforms for arithmetic-intensive workloads whose datasets fit in VRAM.

4.5 Combination Success

LCP degenerates back to TBN when oversubscribing VRAM. This cause severe performance drops and is not contending with mapping the memory in oversubscription scenario. Being interconnect-bound when our workload fits into VRAM cannot compete with pinned memory. So using zero-copy in this case is not an option for performance-sensitive cases.

Table 1 summarizes this trade-off across six qualitative dimensions: performance for workloads larger than VRAM, performance for workloads smaller than VRAM, VRAM locality, migration overhead, implementation complexity, and performance stability. TBN performs poorly under oversubscription but remains competitive when the workload fits into VRAM. Memory mapping shows the opposite behavior: it is more robust under oversubscription, but sacrifices VRAM locality and remains fundamentally interconnect-bound. Our approach achieves the best overall balance across these dimensions. In particular, it performs well in both workload regimes while preserving locality and providing high performance stability. The main trade-off is that it incurs more overhead and requires more programmer involvement than TBN or pure mapping.

HELM, proposed by Jones et al. [24], addresses oversubscription through explicit placement decisions at allocation granularity. Based on the results reported in their paper, HELM improves

substantially over TBN in oversubscribed settings and therefore occupies a stronger middle ground than either TBN or mapping alone. As shown in Table 1, it achieves solid performance across most considered dimensions, although this comes at the cost of increased implementation complexity. It performs fairly well in all dimensions shown in Table 1(excluded complexity). However, its allocation-level decisions can be too coarse for applications with complex or heterogeneous memory access patterns. In contrast, our approach operates at much finer granularity, which provides greater flexibility and enables more precise placement decisions. Jones et al. also propose a static strategy that is conceptually similar to ours, but it still acts at allocation granularity and performs worse in our evaluation. In particular, for the GRAMSCHM application in the UVM benchmark, their static strategy performs worse than TBN, whereas our method outperforms TBN by at least 6× (Fig. 13).

Overall, our method combines the high performance of explicit data movement when the working set fits into VRAM with the robustness of mapped memory under oversubscription. In this sense, it captures the main advantages of both approaches.

5 EXPERIMENTAL EVALUATION

In this section we will evaluate our proposed Framework against a variety of workloads and other approaches like mapping the memory or using explicit data movement.

We will focus on the UVM-Benchmark [21] suite. This benchmark provides a variety of different scientific workloads. For section 5.2 and 5.3, we focus on four of the workloads provided. To get a wide overview over our proposed pattern, we will use patterns with a different memory access pattern. We will cover sequential, ranged and random access patterns. To cover the sequential access pattern, we will use a simple pathfinder application provided. Ranged access pattern is covered by looking at a kmeans implementation. Lastly, to cover random access pattern, we will look at the radix-sort application provided by thrust[8] and our own provided bitonic-sort implementation. The used bitonic-sort implementation is the same one which we shortly introduced in section 3.4.

To understand more deeply how our provided framework performs in other scenarios, we will shortly look into a GPU database implementation, and run some TPC-H queries (section 6). This will deepen our understanding of workloads with low-arithmetic volume which are mainly memory-bound.

In the end, we will focus on the performance of our approach when oversubscribing VRAM. We will compare our approach to the one implemented by CUDA-itself (TBN) and also we will split up our framework and will look at how the different parts of our framework will perform. This will be done in section 5.4.

5.1 Experimental Setup

We used an NVIDIA RTX 5080 based on the blackwell [35] architecture. The NVIDIA RTX 5080 provides 16GiB VRAM and our physical RAM is 32GiB. Our GPU is connected over PCIe 5.0 [37] with x16 width and maximum throughput of 63GB/s. Our CPU is a AMD Ryzen 7 9700X 8-Core Processor with an x86_64 architecture.

Our setup consists of a single GPU-CPU connection. We assume that all data will first be accessed on the CPU and then transferred to the GPU. We will not look at standalone GPU performance, which is

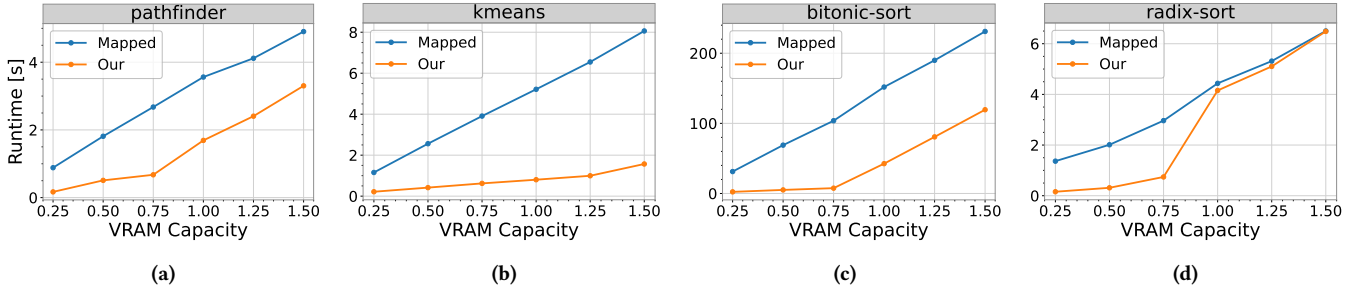


Figure 11: Our Approach against Mapping the Memory. Lower means better. We map the memory by using memory advises.

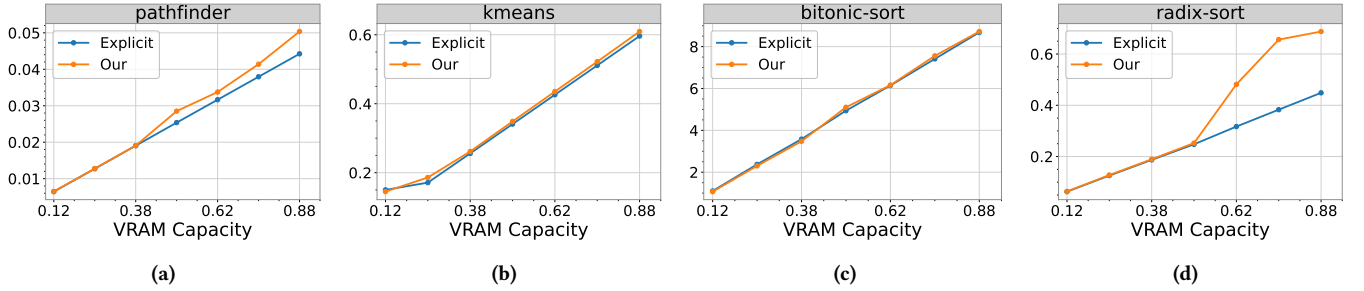


Figure 12: Our Approach against Explicit Data Movement. Lower means better

not meaningful in a discrete GPU setup. We do not assume physical unified memory, and our test cases will focus on CUDA UVM. We use nvcc 12.4 for compilation.

5.2 Is Mapping the Memory enough?

We expect our approach to outperform pure memory mapping, since pages resident in VRAM can be accessed through HBM, while pages that are not placed in VRAM remain mapped. Consequently, the worst-case behavior remains interconnect-bound, while the best case benefits from fast device-local memory access. In Fig. 11, we can see the results of four workloads (pathfinder, kmeans, bitonic-sort, radix-sort). The results are as expected. Our approach performs better than only mapping the memory. Since the UVM allocation updates the page table properly when data is moved to the device, we can see the huge impact that prefetching has on our performance. The overhead of moving the data to the device is for all of these workloads negligible and the direct HBM access leads to superior performance.

Being interconnect-bound is not competitive, especially in our setup with PCIe 5.0. HBM access is faster by orders of magnitude. For the k-means workload, we can see improvements of up to 6.5×. How impactful our approach is is based on how arithmetic-complex the kernel running on the GPU is. In addition to that, it is important to consider, how often we access a certain page on the GPU. In the implemented k-means application, we access single pages more often than for the Pathfinder workload. This is the reason why, in this case, prefetching the page to the GPU and accessing it over device memory instead of the interconnect improves performance by larger orders. For the pathfinder application the overhead of moving the data to the GPU has a higher influence on the runtime.

As third application, we look at bitonic-sort. The access pattern is here even more complex. When we are smaller than VRAM moving the data to the device pays off. We can speedup performance up to 13× by accessing the data over HBM. However, when we are larger than VRAM our algorithm becomes more compute-bound such that the data-movement pay-off does not have the big influence as when being smaller than VRAM. In the end, the speedup is at most 2×. We can conclude from this that LCP is most effective if we access single pages multiple times, but we are not in the compute-bound region.

Lastly, we see the performance for radix sort in Fig. 11 (d). We can see that our performance converges to the mapped memory performance. When our algorithm becomes memory-bound we are limited by mapped-memory performance. The more we oversubscribe VRAM, we see that the data-movement impact becomes less influential. For workloads where we need to access elements which are not on the GPU frequently, our approach is lower bounded by mapping the memory. However, even for these workloads, when we are smaller than VRAM, we still see a massive performance improvement of up to 8× speedup. So just mapping the memory before we know the workload size is not a viable option.

5.3 Can we compete with Explicit Data Movement?

To compare our approach with explicit data movement techniques, we will look at four different workload types. Figure 16 shows the four different workloads: pathfinder, kmeans, bitonic-sort and radix-sort. Since we use explicit data movement and explicit allocation on the GPU, we can only allocate memory up to VRAM size.

Our approach is expected to perform slightly worse than explicit data movement, because memory advice requests are issued through ioctl calls, adding runtime overhead that is absent in a fully manual data-movement scheme. The pathfinder workload (Fig. 16 (a)) performs as expected. When our workload fits in VRAM we can compete with explicit data movement performance. After occupying more than half of VRAM, our approach performs slightly worse. This can be explained, since we use prefetching based on the free memory given by a CUDA API call. This can lead to an underestimation of free space in device memory. So, converging to VRAM size can trigger some data to be read over the interconnect. Additionally, the pathfinder workload runs in a minor millisecond range, so runtime fluctuations can be caused by other factors on the machine.

For the k-means and bitonic-sort (Fig. 16 (b)-(c)) workload, we see the perfect result. Our approach achieves a similar runtime with explicit data movement, and when our workload fits into VRAM. In both cases, we will not lose any performance using our approach compared to explicit data movement.

Lastly, we can see unexpected behavior for the radix-sort workload (Fig. 16 (d)). After occupying more than half of the VRAM, the performance of our approach begins to drop. To test against radix sort, we use the implementation provided by the Thrust library. Since radix sort is implemented out-of-place, we can control only the placement of the input allocation supplied to the function, but not the full allocation behavior of the operation. In particular, using Thrust’s radix sort introduces opaque internal memory management, which makes part of the allocation strategy effectively a black box. This will lead to unpredictable runtime performance. We can still see that before the VRAM is half-occupied, we achieved competitive performance. Even in cases where the allocation is not fully transparent, our approach can achieve competitive performance and stable performance by drops of only up to 40%. However, if the allocation were fully transparent, we could achieve competitive performance here as well.

In conclusion, our approach performs competitively against explicit data movement when our workload fits into VRAM. Explicit data movement is the holy grail for the provided workloads with sizes smaller than VRAM. Additionally, our approach allows us to oversubscribe in a simple manner, while oversubscribing using explicit data movement has to be done by hand and will lead to complex manual code changes.

5.4 Variety of Workloads

To evaluate whether our approach delivers stable performance across the full workload spectrum, we look at eight workloads ranging from sequential to random access patterns.

To prove that our approach stabilizes performance, we look at workload sizes 1.5× of VRAM size. We compare our combined LCP and zero-copy strategy against TBN alone, and against each component in isolation, to quantify the contribution of each part of the framework.

Our approach consists of two parts. First, we use LCP to get competitive performance when our workload size does not exceed VRAM. Second, we use zero-copy memory management such that

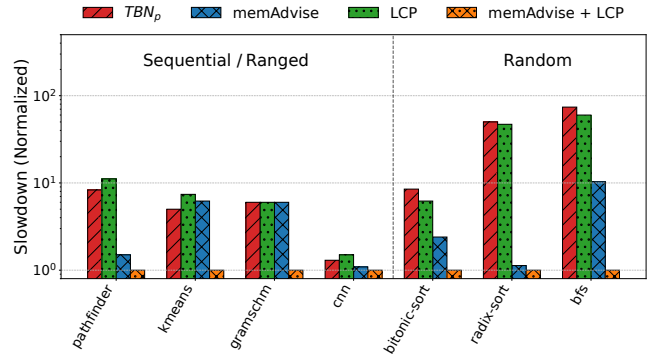


Figure 13: Different Prefetching mechanisms compared with memAdvise. Workload size is 1.5x of VRAM. All runtimes are normalized against the memAdvise + Chunk-Prefetching as baseline. Lower means better. All workloads except for the sorting ones are based on the UVM-benchmark [21]

we will not run into expensive on-demand page faulting and frequent evictions.

Figure 13 shows the result of the experiment. We can see that for sequential/range-based workloads, the performance drop is not as sharp as for random workloads. When looking at the CNN workload, the performance drop is only marginal, with an increase of roughly 30%. This can be explained, since the CNN here is a really small neural network, and the main memory contributor is the training set. Since going through the training set is a linear access pattern, a lot of latency is hidden there.

However, looking at the random workloads displays the limitations of TBN-prefetching. The performance for BFS can drop up to 74×. ero-copy is essential here: by eliminating migration and eviction entirely, we improve the performance by orders of magnitude. The same pattern can be seen in radix sort. Using the TBN-prefetching performance drops by 50×. Here, LCP cannot stabilize the performance. The on-demand page faulting is still dominating the runtime and causes a severe performance drop.

6 EVALUATION ON GPU DATABASE WORKLOADS.

Several works [25, 42] have introduced libraries for executing database query operators on GPUs. While their standalone GPU performance is impressive, data movement becomes the primary bottleneck, rendering execution memory-bound. TPC-H queries typically exhibit low arithmetic intensity, meaning that the amount of computation performed per byte transferred is small. In addition, many of these operators stream over large relations and revisit individual memory pages only infrequently. Consequently, page migrations into VRAM provide limited opportunity for reuse, so data-movement overhead can dominate execution time. To evaluate the effectiveness of our approach for such workloads, we analyze TPC-H queries executed on the GPU using the Crystal library [42].

Figure 14 presents the runtime results for several TPC-H queries at a scale factor of 100, where the dataset fully fits into VRAM. In this setting, transferring data to device memory and accessing it

via HBM is beneficial. Our approach achieves the best performance compared to both zero-copy and the standard TBN prefetcher. The CUDA-implemented TBN prefetcher performs worst, indicating that on-demand page migration is ineffective for memory-bound workloads.

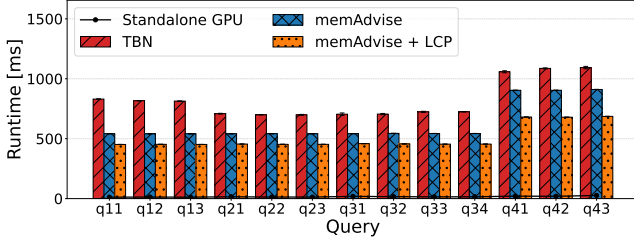


Figure 14: Comparison on the Star Scheme Benchmark (SSB) [36]. Scaling Factor of 100 for the benchmark.

Figure 15 shows results for the same benchmark at a scale factor of 150, where the dataset exceeds the available VRAM capacity, leading to oversubscription. Queries with an identifier greater than 40 require additional allocations and cannot be executed under the current system configuration. In this scenario, avoiding data migration and relying directly on zero-copy achieves the best performance. Pure zero-copy outperforms our approach by approximately 15%, while TBN again performs worst across all queries. These results further confirm that on-demand page faulting is unsuitable for oversubscribed, memory-bound workloads.

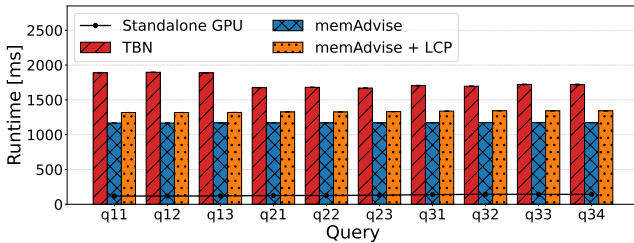


Figure 15: Comparison on the Star Scheme Benchmark (SSB) [36]. Scaling Factor of 150 for the benchmark.

In summary, for workloads with low arithmetic intensity and limited page reuse, zero-copy can be the most effective approach. However, when the dataset fits into VRAM, our method outperforms pure zero-copy. Overall, while there is no universally optimal solution between zero-copy and our approach, the proposed method consistently outperforms TBN across workloads of all sizes.

7 DOES AI KNOW HOW TO USE UVM?

To illustrate that effective UVM optimization requires domain knowledge that general-purpose LLMs currently lack, we evaluate whether state-of-the-art code generation models can discover our optimization strategy from first principles.

- (1) "Use Managed Memory."
- (2) "Use Managed Memory and optimize it."

- (3) "Use Managed Memory. Prefetch as much data as possible to the GPU. Prefetch from most important to least important data to the GPU. Never prefetch more than GPU VRAM available. For every managed allocation use memory advises to give him a zero-copy hint. setPreferredLocation should be CPU and setAccessedBy should by GPU."

For our experiment, we used Google Gemini 3 in thinking mode. We started each chat from scratch, with only the file and its corresponding prompt. In Fig. 16, we can see the results. We gave the chatbot k-means and a pathfinder implementation based on the UVM benchmark suite. Both implementations used pinned memory and employed explicit data movement. We can see in Fig. 16 (a) that the specific prompt based on our findings improved performance hugely.

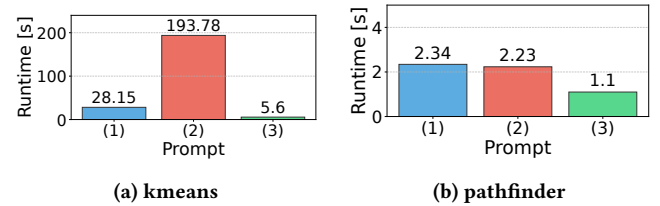


Figure 16: AI Prompt comparison. Workload size 1.5x VRAM.

Additionally, we can see that letting the LLM optimize the managed code on its own can cause serious performance degradation. The performance between letting LLM optimize the code (2) and our optimization (3) is roughly 34x. This also shows one of UVM's recurring problems. When using prefetching and memory advises the wrong way, UVM can cause critical performance drops. The observed failure modes (wrong advice selection, unbounded prefetching, and allocation-agnostic migration) are precisely the limitations our framework's three-rule prefetching scheme is designed to prevent, further motivating the need for explicit, principled guidelines.

In Figure 16 (b), we can see that our prompt is still performing the best. However, we see that the performance difference is not as huge as for k-means. One of the reasons is that the pathfinder implementation has far fewer allocations, and so far fewer points to optimize or error-prune. Nevertheless, our prompt still outperforms the LLM optimizations.

Another interesting finding is that for both tests, when we let the LLM optimize UVM by itself it tried to use the memory advise setReadMostly. It did not try to use any other of the memory advises. The LLM also understands that prefetching is important and adds it at appropriate places. However, it does not understand that we should only prefetch as much data as space is available in device memory. It also does not differentiate between different allocations; it just tries to prefetch everything to the GPU.

8 PROGRAMMING GUIDELINES

In this section, we present our fourth main contribution: a **reproducible programming framework** that stabilizes Unified Memory performance under oversubscription. Our framework eliminates the pathological behavior of naive UVM and consistently delivers predictable execution across workloads. Additionally, our

framework remains simple, preserving the abstraction benefits of UVM. We did our testing using the CUDA API, so we will refer to CUDA API calls here. However, the general idea can also be used with other vendors and should be used. Syntax has to be adjusted accordingly.

Older programming guidelines [27] do not check against memory advises only looked at how we should prefetch.

8.1 Zero-Copy Advice

Every allocation will be given a zero-copy advice. One advantage of using managed memory in CUDA is that if the data is available in device memory the page table will be updated properly. Such that we access the data over HBM if possible which will outperform the interconnect latency. Giving managed memory the zero-copy advice can be done fairly simple as given in the following example:

```
...
cudaMallocManaged(&ptr, sz);

cudaMemAdvise(ptr, sz, cudaMemAdviseSetPreferredLocation, cpu_id);
cudaMemAdvise(ptr, sz, cudaMemAdviseSetAccessedBy, gpu_id);
...

...
cudaMallocManaged(&ptr, sz);

cudaMemGetInfo(&total_m, &free_m)
cudaMemPrefetchAsync(ptr, sz > free_m? free_m : sz, gpu_id, 0, 0);
...
```

This syntax is based on the NVIDIA blogs[15, 41]. The compiler will know that it has to read the bytes over the interconnect and the page table is populated properly.

8.2 Prefetching

Our prefetching guideline consists of three simple rules:

- (1) From Most important to least important
- (2) Prefetch as much concurrently as possible
- (3) Prefetch should not exceed VRAM

Important is defined as how often do we access certain data. Prefetch as much data as possible can be understand, that we never split up a certain allocation if it is possible to prefetch the whole allocation size we will migrate it to the GPU. However, the size we prefetch should never exceed available GPU memory since we then would start trashing the device memory and page-fault handling would decrease our performance.

Figure 17 shows us a flowchart on how we should manage our allocations to follow the provided principled framework. As an initial step before we start the prefetching, we map all allocations using the correct memory advises. Then, at (1) we order our allocations based on most important data to least important data. After that, we look at the most important allocation (2) and check if the GPU has free space available. If yes, we (3) prefetch as much data as possible. So we take the minimum between free space available in device memory and allocation size. Then we remove the allocation from our list (4). If in our allocation stack are still elements left, we will start again at (2) and look at the next allocation. If our stack is empty, we terminate the prefetching and continue with the rest of our implementation.

This flow chart properly handles the ordering from most important to least important data (1). Through an initial ordering. It

also makes sure that the third rule is not violated by prefetching maximal the free space on the device memory (3). This ensures that we page trashing does not start. By prefetching as much as possible we also make sure that the second rule is carried out accordingly.

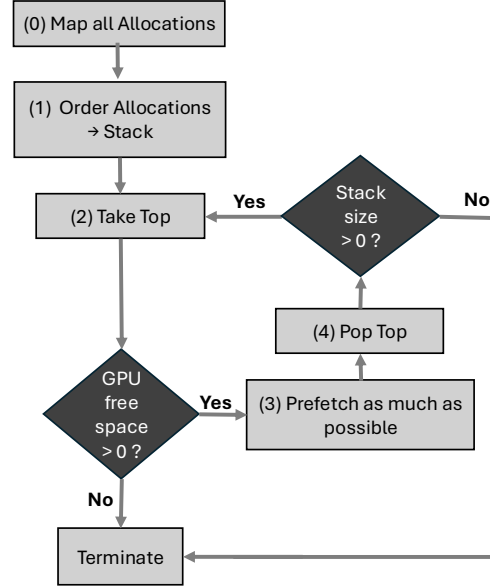


Figure 17: Flowchart for principled framework.

9 CONCLUSION & OUTLOOK

Our work showed that it is possible exploit UVM as a hybrid memory management model and stabilize application performance regardless of the workload size and type. Aggressive prefetching combined with mapping the CPU memory can give us a predictable and stable performance. One limitation of the presented approach is that there are still applications where it is possible that other strategies can outperform us. However our main goal was to make the approach stable, general and simple, instead of a workload-specific strategy.

Future work could implement a hardware-based prefetcher which follows the provided prefetching rules. This hardware-prefetcher could enable wider UVM adoption. Our work primarily focused on a single CPU-GPU setup, so another open question is how multi-GPU setups[2, 12, 47] infer with our approach. Future work could implement a hardware-based prefetcher which follows the provided prefetching rules. This hardware-prefetcher could enable wider UVM adoption. Our work primarily focused on a single CPU-GPU setup, so another open question is how multi-GPU setups[2, 12, 47] infer with our approach.

REFERENCES

- [1] NVIDIA CORPORATION. 2016. GP100 Pascal Whitepaper. (2016). <https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf>
- [2] Tyler Allen, Bennett Cooper, and Rong Ge. 2024. Fine-grain Quantitative Analysis of Demand Paging in Unified Virtual Memory. *ACM Transactions on Architecture and Code Optimization* 21, 1 (March 2024), 1–24. <https://doi.org/10.1145/3632953>

- [3] Tyler Allen and Rong Ge. 2021. Demystifying GPU UVM Cost with Deep Runtime and Workload Analysis. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 141–150. <https://doi.org/10.1109/IPDPS49936.2021.00023>
- [4] Tyler Allen and Rong Ge. 2021. In-depth analyses of unified virtual memory system for GPU accelerated computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, St. Louis Missouri, 1–15. <https://doi.org/10.1145/3458817.3480855>
- [5] Amazon Web Services. 2026. Accelerated Computing Instances. <https://aws.amazon.com/ec2/instance-types/accelerated-computing/>. Accessed: 2026-04-01.
- [6] Ammar Ahmad Awan, Ching-Hsiang Chu, Hari Subramoni, Xiaoyi Lu, and Dhabaleswar K. Panda. 2018. OC-DNN: Exploiting Advanced Unified Memory Capabilities in CUDA 9 and Volta GPUs for Out-of-Core DNN Training. In *2018 IEEE 25th International Conference on High Performance Computing (HiPC)*. IEEE, Bengaluru, India, 143–152. <https://doi.org/10.1109/HiPC.2018.00024>
- [7] Kevin Beardslee et al. 2023. Grand Teton: Meta’s Next-Generation AI Training Platform. In *Proceedings of the 2023 Open Compute Project (OCP) Global Summit*. <https://www.opencompute.org/> Describes the shift to discrete high-power GPU modules to meet LLM demands.
- [8] Nathan Bell and Jared Hoberock. 2012. Thrust: A Productivity-Oriented Library for CUDA. In *GPU Computing Gems Jade Edition*. Morgan Kaufmann.
- [9] Berenger Bramas. 2017. Fast Sorting Algorithms using AVX-512 on Intel Knights Landing. (2017).
- [10] Jatin Chhugani, Anthony D. Nguyen, Victor W. Lee, William Macy, Mostafa Hagog, Yen-Kuang Chen, Akram Baransi, Sanjeev Kumar, and Pradeep Dubey. 2008. Efficient implementation of sorting on multi-core SIMD CPU architecture. *Proceedings of the VLDB Endowment* 1, 2 (Aug. 2008), 1313–1324. <https://doi.org/10.14778/1454159.1454171>
- [11] Steven Chien, Ivy Peng, and Stefano Markidis. 2019. Performance Evaluation of Advanced Features in CUDA Unified Memory. In *2019 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*. IEEE, Denver, CO, USA, 50–57. <https://doi.org/10.1109/MCHPC49590.2019.00014>
- [12] Sangjin Choi, Taeksoo Kim, Jinwoo Jeong, Myeongjae Jeon, Youngjin Kwon, and Jeongseob Ahn. 2022. Memory Harvesting in Multi-GPU Systems with Hierarchical Unified Virtual Memory. (2022).
- [13] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. 2019. Interplay between hardware prefetcher and page eviction policy in CPU-GPU unified virtual memory. In *Proceedings of the 46th International Symposium on Computer Architecture*. ACM, Phoenix Arizona, 224–235. <https://doi.org/10.1145/3307650.3322224>
- [14] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. 2020. Adaptive Page Migration for Irregular Data-intensive Applications under GPU Memory Oversubscription. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, New Orleans, LA, USA, 451–461. <https://doi.org/10.1109/IPDPS47924.2020.00054>
- [15] Chirayu Garg and Nikolay Sakharlykh. 2021. Improving GPU Memory Oversubscription Performance. <https://developer.nvidia.com/blog/improving-gpu-memory-oversubscription-performance/>. Accessed: 2026-03-02.
- [16] Buundefinedra Gedik, Rajesh R. Bordawekar, and Philip S. Yu. 2007. CellSort: high performance sorting on the cell processor. In *Proceedings of the 33rd International Conference on Very Large Data Bases (Vienna, Austria) (VLDB ’07)*. VLDB Endowment, 1286–1297.
- [17] Prasun Gera, Hyojong Kim, Piyush Sao, Hyesoon Kim, and David Bader. 2020. Traversing large graphs on GPUs with unified memory. *Proceedings of the VLDB Endowment* 13, 7 (March 2020), 1119–1133. <https://doi.org/10.14778/3384345.3384358>
- [18] Seokjin Go, Hyunwuk Lee, Junsung Kim, Jiwon Lee, Myung Kuk Yoon, and Won Woo Ro. 2023. Early-Adaptor: An Adaptive Framework for Proactive UVM Memory Management. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, Raleigh, NC, USA, 248–258. <https://doi.org/10.1109/ISPASS57527.2023.00032>
- [19] Google Cloud. 2026. About GPU Instances. <https://cloud.google.com/compute/docs/gpus/about-gpus>. Accessed: 2026-04-01.
- [20] Michael Gowanlock and Ben Karsin. 2019. A hybrid CPU/GPU approach for optimizing sorting throughput. *Parallel Comput.* 85 (July 2019), 45–55. <https://doi.org/10.1016/j.parco.2019.01.004>
- [21] Yongbin Gu, Wenxuan Wu, Yunfan Li, and Lizhong Chen. 2020. UVMBench: A Comprehensive Benchmark Suite for Researching Unified Virtual Memory in GPUs. <https://doi.org/10.48550/arXiv.2007.09822> arXiv:2007.09822 [cs].
- [22] Zezhou Huang, Krystian Sakowski, Hans Lehnert, Wei Cui, Carlo Curino, Matteo Interlandi, Marius Dumitru, and Rathijit Sen. [n.d.]. GPU Acceleration of SQL Analytics on Compressed Data. ([n.d.]).
- [23] M.F. Ionescu and K.E. Schauer. 1997. Optimizing parallel bitonic sort. In *Proceedings 11th International Parallel Processing Symposium*. IEEE Comput. Soc. Press, Geneva, Switzerland, 303–309. <https://doi.org/10.1109/IPPS.1997.580914>
- [24] Nathan Gens, Tyler Allen, and Rong Ge. 2025. HELM: Characterizing Unified Memory Accesses to Improve GPU Performance under Memory Oversubscription. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, St. Louis MO USA, 490–504. <https://doi.org/10.1145/3712285.3759812>
- [25] Marko Kabić, Shriram Chandran, and Gustavo Alonso. 2025. Maximus: A Modular Accelerated Query Engine for Data Analytics on Heterogeneous Systems. *Proceedings of the ACM on Management of Data* 3, 3 (June 2025), 1–25. <https://doi.org/10.1145/3725324>
- [26] Hyojong Kim, Jaewoong Sim, Prasun Gera, Ramyad Hadidi, and Hyesoon Kim. 2020. Batch-Aware Unified Memory Management in GPUs for Irregular Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS ’20)*. Association for Computing Machinery, New York, NY, USA, 1357–1370. <https://doi.org/10.1145/3373376.3378529>
- [27] Marcin Knap and Paweł Czarnul. 2019. Performance evaluation of Unified Memory with prefetching and oversubscription for selected parallel CUDA applications on NVIDIA Pascal and Volta GPUs. *The Journal of Supercomputing* 75, 11 (Nov. 2019), 7625–7645. <https://doi.org/10.1007/s11227-019-02966-8>
- [28] Raphael Landaverde, Tiansheng Zhang, Ayse K. Coskun, and Martin Herbordt. 2014. An investigation of Unified Memory Access performance in CUDA. In *2014 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, Waltham, MA, USA, 1–6. <https://doi.org/10.1109/HPEC.2014.7040988>
- [29] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [30] Clemens Lutz, Sebastian Breß, Steffen Zeuch, Tilmann Rabl, and Volker Markl. 2020. Pump Up the Volume: Processing Large Data on GPUs with Fast Interconnects. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 1633–1649. <https://doi.org/10.1145/3318464.3389705>
- [31] K. V. Manian, A. A. Ammar, A. Ruhela, C.-H. Chu, H. Subramoni, and D. K. Panda. 2019. Characterizing CUDA Unified Memory (UM)-Aware MPI Designs on Modern GPU Architectures. In *Proceedings of the 12th Workshop on General Purpose Processing Using GPUs*. ACM, Providence RI USA, 43–52. <https://doi.org/10.1145/3300053.3319419>
- [32] Seung Won Min, Vikram Sharma Malthody, Zaid Qureshi, Jinjun Xiong, Eiman Ebrahimi, and Wen-mei Hwu. 2020. EMOG: efficient memory-access for out-of-memory graph-traversal in GPUs. *Proceedings of the VLDB Endowment* 14, 2 (Oct. 2020), 114–127. <https://doi.org/10.14778/3425879.3425883>
- [33] Alok Mishra, Lingda Li, Martin Kong, Hal Finkel, and Barbara Chapman. 2017. Benchmarking and Evaluating Unified Memory for OpenMP GPU Offloading. In *Proceedings of the Fourth Workshop on the LLVM Compiler Infrastructure in HPC*. ACM, Denver CO USA, 1–10. <https://doi.org/10.1145/3148173.3148184>
- [34] NVIDIA. 2026. CUDA Runtime API: Memory Management. https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html. CUDA Toolkit v13.2.0, Last updated March 6, 2026, Accessed: 2026-04-02.
- [35] NVIDIA Corporation. 2025. NVIDIA RTX Blackwell GPU Architecture. White paper. NVIDIA. <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>
- [36] Pat O’Neil, Betty O’Neil, and Xuedong Chen. [n.d.]. 1. Star Schema Based on TPC-H. ([n.d.]).
- [37] PCI-SIG. 2019. *PCI Express Base Specification Revision 5.0, Version 1.0*. Technical Report. PCI-SIG. <https://pcisig.com/specifications/pciexpress>
- [38] Hagen Peters, Ole Schulz-Hildebrandt, and Norbert Luttenberger. 2011. Fast in-place, comparison-based sorting with CUDA: a study with bitonic sort. *Concurrency and Computation: Practice and Experience* 23, 7 (May 2011), 681–693. <https://doi.org/10.1002/cpe.1686>
- [39] PyTorch Contributors. 2024. Unified Memory oversubscription discussion. <https://github.com/pytorch/pytorch/issues/158122>. GitHub issue.
- [40] Amir Hossein Nodehi Sabet, Zhijia Zhao, and Rajiv Gupta. 2020. Subway: minimizing data transfer during out-of-GPU-memory graph processing. In *Proceedings of the Fifteenth European Conference on Computer Systems*. ACM, Heraklion Greece, 1–16. <https://doi.org/10.1145/3342195.3387537>
- [41] Nikolay Sakharlykh. 2017. Maximizing Unified Memory Performance in CUDA. <https://developer.nvidia.com/blog/maximizing-unified-memory-performance-cuda/>. Accessed: 2026-03-02.
- [42] Anil Shanbhag, Samuel Madden, and Xiangyao Yu. 2020. A Study of the Fundamental Performance Characteristics of GPUs and CPUs for Database Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 1617–1632. <https://doi.org/10.1145/3318464.3380595>
- [43] Anil Shanbhag, Holger Pirk, and Samuel Madden. 2018. Efficient Top-K Query Processing on Massively Parallel Hardware. In *Proceedings of the 2018 International Conference on Management of Data*. ACM, Houston TX USA, 1557–1570. <https://doi.org/10.1145/3183713.3183735>
- [44] TOP500. 2025. Highlights – November 2025. <https://top500.org/lists/top500/2025/11/highs/>. Accessed: 2026-04-01.
- [45] Jan Vesely, Arkaprava Basu, Mark Oskin, Gabriel H. Loh, and Abhishek Bhat-tacharjee. 2016. Observations and opportunities in architecting shared virtual memory for heterogeneous systems. In *2016 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, Uppsala, Sweden, 161–171. <https://doi.org/10.1109/ISPASS.2016.7482091>

- [46] Qi Yu, Bruce Childers, Libo Huang, Cheng Qian, and Zhiying Wang. 2020. A quantitative evaluation of unified memory in GPUs. *The Journal of Supercomputing* 76, 4 (April 2020), 2958–2985. <https://doi.org/10.1007/s11227-019-03079-y>
- [47] Amir Kavyan Ziabari, Yifan Sun, Yenai Ma, Dana Schaa, José L. Abellán, Rafael Ubal, John Kim, Ajay Joshi, and David Kaeli. 2016. UMH: A Hardware-Based Unified Memory Hierarchy for Systems with Multiple Discrete GPUs. *ACM Transactions on Architecture and Code Optimization* 13, 4 (Dec. 2016), 1–25. <https://doi.org/10.1145/2996190>